

ST67W61 Bare-metal implementation

- [ST67W61 Bare-metal implementation](#)
 - [1. Supported boards](#)
 - [2. Projects functionality](#)
 - [2.1 Echo](#)
 - [2.2 HTTP_BLE_Heartrate](#)
 - [2.3 MQTT_BLE_Commissioning](#)
 - [3. SPI driver explanation](#)
 - [3.1 Communication timing](#)
 - [3.2 Low-Level SPI Frame Format](#)
 - [3.3 Command–Response Model](#)
 - [3.4 Data Sending Model](#)
 - [3.5 Handling Out-of-Band Reports](#)
 - [3.6 Implementation Details](#)
 - [5. Porting to other boards](#)
 - [6. Known limitations](#)

Extension of the basic X-CUBE-ST67W61 package. It demonstrates a simplified implementation of the AT protocol (over SPI) for the Wi-Fi and BLE NCP module.

Compared to the base package, this implementation does not use FreeRTOS and the Middleware layer. The AT commands are sent over SPI in blocking mode - without DMA or interrupts.

The application layer needs to simply pass a string containing the AT command with relevant parameters to the interface function and wait for the return value - then the response can be checked to evaluate success / failure of the command.

The list of supported commands, responses and reports can be found on the [Wiki](#) page.

After compilation (using LL drivers, CubelIDE, space optimization), memory consumption of approximately 22 kB of Flash and 3.5 kB of RAM can be achieved - with application code included.

This is shown in the MQTT_BLE_Commissioning application.

1. Supported boards

The driver is ported to these boards (with the given example applications):

- NUCLEO-U575ZI-Q
 - Echo
 - HTTP_BLE_Heartrate
- NUCLEO-G0B1RE
 - Echo
 - MQTT_BLE_Commissioning
- NUCLEO-C031C6
 - MQTT_BLE_Commissioning

See [Porting to other boards](#) for details on pinout etc, and how to port to other Nucleo boards.

2. Projects functionality

All projects have the following functionality implemented in the user-application level code:

- UART is used for logging to the terminal of the PC connected to the host (through the printf() function).
- Incoming report messages (such as network scan results, connection, ping) are printed to the terminal.

2.1 Echo

Wifi is initialized in station mode.

Network scan is done to find available Wi-Fi networks.

It connects to an access point - with SSID and password specified in the code.

Lastly, it pings a website to verify the connection and disconnects from the network.

2.2 HTTP_BLE_Heartrate

Wifi is initialized in station mode and connects to an access point - with SSID and password specified in the code.

The assigned IP address is printed to the terminal.

BLE is initialized in server role and the Heartrate service with characteristics is created.

Using an interrupt from a HW timer, dummy data is periodically updated in global variables.

When a web client (web browser on a phone) connects to the Wifi station's IP address, the application sends an HTTP page displaying graphs for heartrate and calories consumed.

The HTTP code contains instruction to periodically request new values.

The web client sends these requests and the application responds with current values.

After a BLE client (e.g. BLE Toolbox application) connects, it can view the current state of the BLE characteristics with values being updated automatically by the application.

BLE and Wifi connection can be mantained in parallel.

2.3 MQTT_BLE_Commissioning

Wifi is initialized in station mode.

BLE is initialized in server role and the BLE commissioning service with characteristics is created.

The application tries to connect to a previously saved access point.

If not successful, it waits for the Wifi parameters to be set through the BLE characteristic (using BLE Toolbox application).

If successfull, the user can still connect through BLE and change to a different Wifi network.

After Wifi connection, MQTT is initialized and connected to a broker.

The IoT MQTT Panel application (available for [Android](#) and [iOS](#)) can then be used to connect to the same broker.

With the configuration file provided in the MQTT_Panel_config folder, the IoT MQTT Panel application can receive RSSI statistics and send data which mimic a controlling the level and color of an LED light.

Incoming messages are parsed and displayed on the terminal.

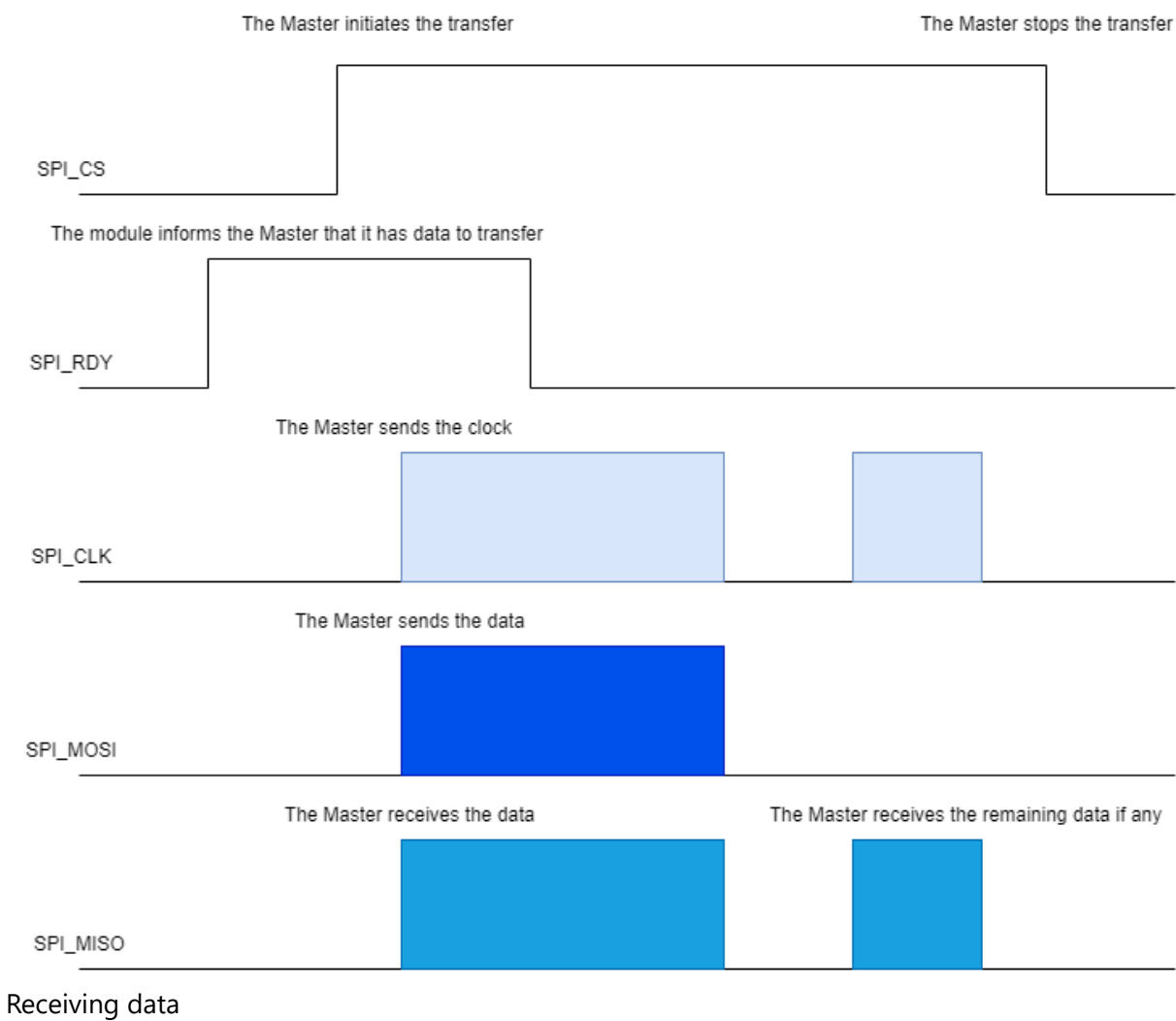
3. SPI driver explanation

3.1 Communication timing

After setting the Chip Enable pin high, the host needs to wait for the NCP to initialise the first transfer.

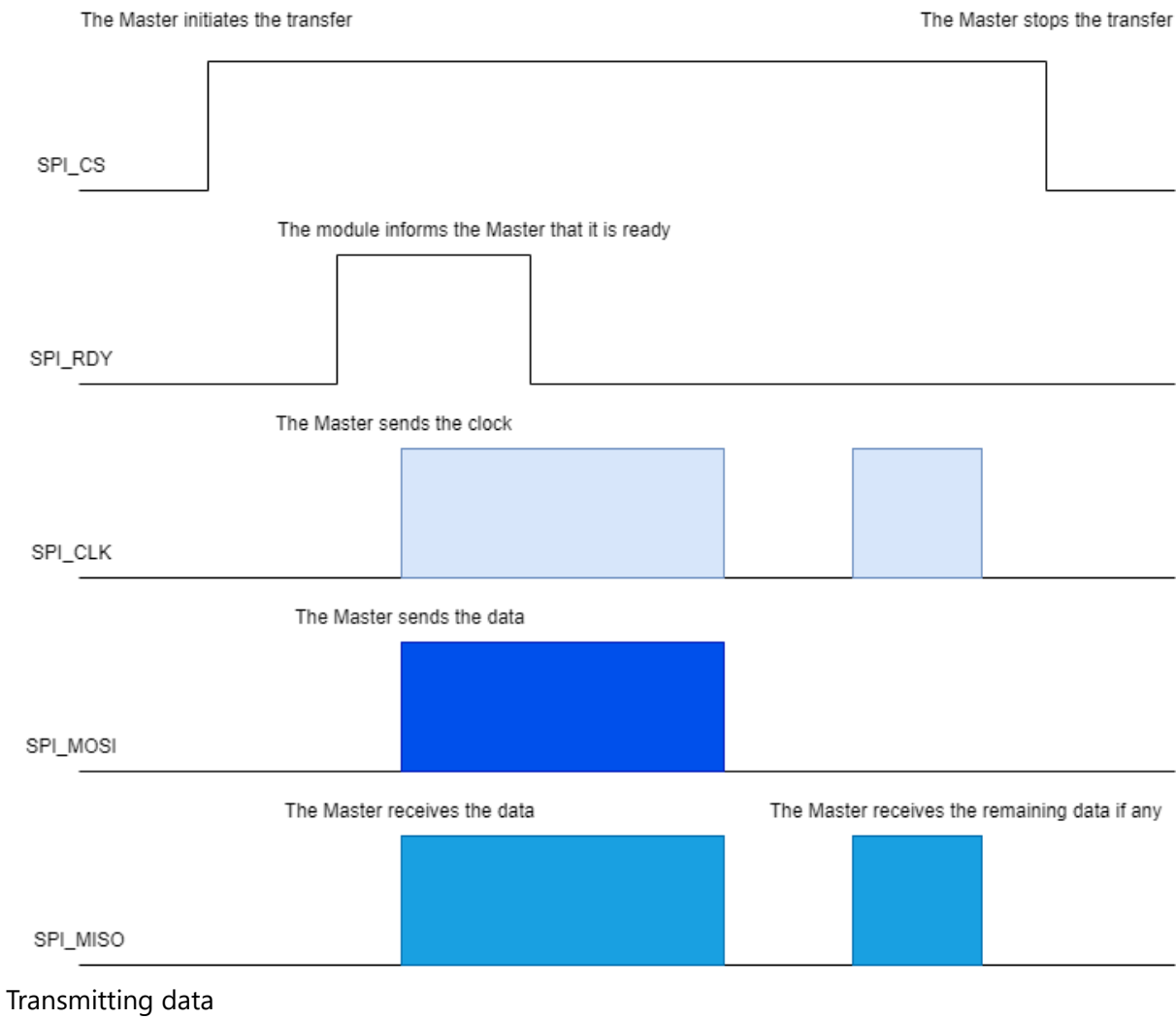
The first data transmitted by the NCP will be **ready**.

When the NCP has data to transfer, it raises the RDY signal/interrupt to inform the host, which in turn raises CS and generates clock in order to receive the data.



When sending data, the host needs to raise the CS pin and wait for the NCP to raise the RDY pin.

After that, the host generates clock and sends its data.

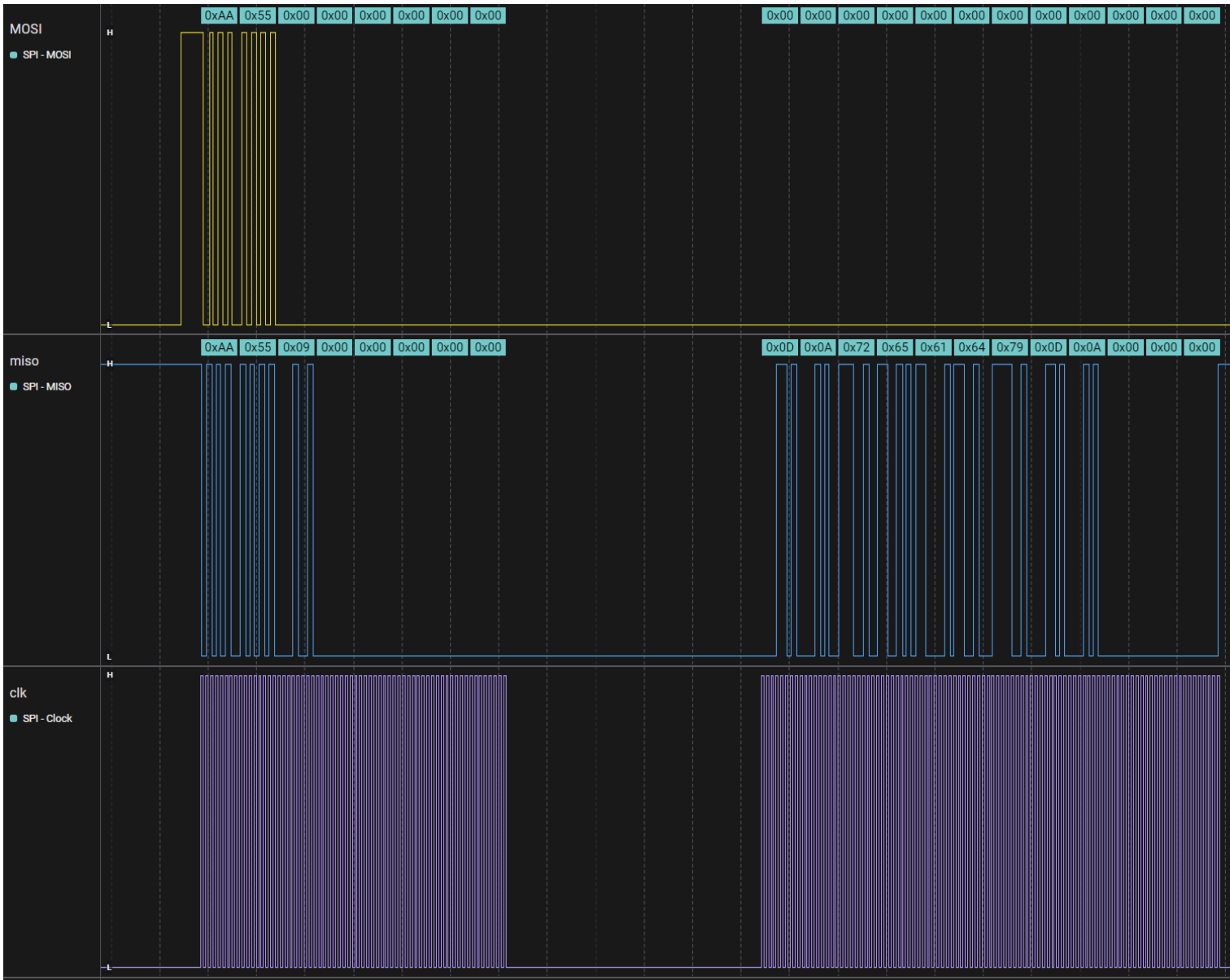


3.2 Low-Level SPI Frame Format

There are two possible formats for any communication with the NCP. Each time communication is established (from either side), the first (header) frame transmitted will have the following format:

Field	Size (bytes)	Description
Magic Code	2	Fixed value 0x55AA identifying a valid SPI frame, starting with the lower byte
Data Length	2	Length of the payload (in bytes), starting with the lower byte
Reserved / Padding	4	Reserved for alignment or protocol extension

Then the initiating side will transmit the specified number of data bytes in the following frame. This frame contains only data - without any leading header. If the length of the data is not 4-byte aligned, the frame needs to be padded by 0x88 data. In such a case, the length specified in the header frame is still the number of valid data bytes (without padding). This is illustrated in the figure below.



Header and data frame

Example (sending "AT" and receiving "OK"):

```
→ CS High
← RDY High
→ 0xAA | 0x55 | 0x04 | 0x00 | 0x00 | 0x00 | 0x00 | 0x00
→ 0x41 | 0x54 | 0x0D | 0x0A
=   A  |   T  |  \r  |  \n
← 0xAA | 0x55 | 0x06 | 0x00 | 0x00 | 0x00 | 0x00 | 0x00
← 0x0D | 0x0A | 0x4F | 0x48 | 0x0D | 0x0A | 0x00 | 0x00
=  \r  |  \n  |   O  |   K  |  \r  |  \n
→ CS Low
← RDY Low
```

3.3 Command–Response Model

Commands are ASCII AT-style strings ending with `\r\n`. Each command generates an immediate response, indicating the success / failure of the command. Responses are also terminated with `\r\n`.

Command Type	Example	Description
SET	<code>AT+CMD=<val></code>	Set configuration
QUERY	<code>AT+CMD?</code>	Get parameter
EXECUTE	<code>AT+CMD</code>	Execute action

SET and EXECUTE commands typically generate a single response, such as `OK` or `ERROR`. Some commands (described in the [following section](#)) additionally produce `>` which indicates that the NCP is ready to receive additional data frame. For QUERY commands, the response will be composed of two parts: a response with data corresponding to the query followed by `OK`. `busy p` response means that the NCP is busy but it will continue with execution and response in the next frame.

Response	Meaning
<code>OK</code>	Command success
<code>ERROR</code>	Command or syntax error
<code>busy p</code>	Device not ready
<code>></code>	Ready to receive binary data
<code>+CMD:<val></code>	Query result

Example:

```
→ AT+BLEINIT?  
← busy p  
← +BLEINIT:0  
← OK
```

3.4 Data Sending Model

Some of the AT commands require the host to send data in a separate SPI frame. These are typically for writing into the FLASH or EFUSE region, sending data on a UDP / TCP port, publishing data on MQTT or updating BLE characteristics data. After receiving one of these commands, the NCP will respond with **OK**>. The host is then expected to transmit the data (without the header frame) with length specified in the command frame. The NCP will confirm reception of the data by either of the following responses:

Response	Meaning
SEND OK	Data sent successfully
SEND FAIL	Data failed

Example:

```
→ AT+CIPSEND=5  
← OK  
← >  
→ HELLO  
← SEND OK
```

3.5 Handling Out-of-Band Reports

Some commands, such as Wifi scan, will produce the initial **OK** response and the NCP will later send the result of the operation in a separate report. User actions, such as receiving data from a connected device, will also elicit a report. These reports can generally come at any time, even between a command and its response.

Example:

```
→ AT+CWQAP  
+CW:DISCONNECTED  
+CW:ERROR,19  
← OK
```

Example:

```
→ AT+CWLAP  
← OK  
...  
...  
← +CWLAP: ....  
← +CWLAP: ....  
← +CWLAP: ....  
← +CW:SCAN_DONE
```

3.6 Implementation Details

The file spi_iface.c handles the above functionality from adding trailing **\r\n** characters to sending commands and waiting for responses. It should be initialized by calling **spi_iface_init()** with a pointer to an application callback to handle report messages. RDY pin setting and reception of the first **ready** report is handled in this function, as well as testing the interface by sending the first AT command.

Sending commands is possible by calling **spi_iface_command()** with an ASCII string as an argument. When sending a command, the application waits in a blocking loop until a response is received. An integer status is returned to quickly verify success / failure, based on response timeout, **ERROR** response, etc.

All incoming reports (both during command / data exchange and out of this transaction) are passed to the main application to be handled. Both responses and reports are returned as pointers to dynamically allocated memory. It is then the responsibility of the main application to free this memory (after handling the data as necessary) to avoid memory overflows.

spi_port.c then handles the low-level SPI transactions.

The driver is prepared for both HAL and LL driver integration, based on pre-compiler flags.

File	Role
spi_iface.c	SPI frame handling, command/data TX, response parsing
spi_port.c	Hardware SPI interface and GPIO control

Key Functions

Function	Description
spi_iface_init()	Initialize the NCP
spi_iface_command()	Send a command
spi_iface_data()	Send data without header
wifi_iface_report_cb()	Application callback

Additional logging can be enabled to show bytes on the SPI layer.

Outgoing AT commands and incoming replies and reports can also be logged in string format.

Following is a list of files which are important for understanding the application flow and for potentially designing a more advanced driver.

- main.c
 - MX_SPI1_Init - initialization of SPI (communication with the NCP).
 - MX_USART2_UART_Init - initialization of UART (logging to terminal).
 - MX_GPIO_Init - pin initialization.
 - HAL_GPIO_EXTI_Rising_Callback and HAL_GPIO_EXTI_Falling_Callback - handling the external rising and falling SPI_RDY interrupts.
- stm32g0xx_hal_msp.c
 - Initialization of SPI and UART pins.
- stm32g0xx_it.c
 - Interface from system interrupt handler to main application.
- spi_port.h
 - #define NCP_SPI_HANDLE - specify the SPI peripheral handle to be used.
 - #define SPI_TRANSMIT_TIMEOUT_MS - maximum time (in ms) for the SPI transaction timeout.
 - #define SPI_PHY_LOG_CHARS - used to specify the maximum length buffer used of the string used for logging the bytes exchanged over SPI. Set to 0 to disable this log.
- spi_port.c
 - spi_port_init and spi_port_deinit - pulls the CHIP_EN high / low to turn the NCP on / off.
 - spi_port_set_cs - sets the SPI_CS pin before / after the SPI transaction.
 - spi_port_transfer - handles the SPI transaction.
 - spi_port_is_ready - checks the state of the SPI_RDY pin.
- spi_iface.h
 - #define SPI_IFACE_LOG_OUTGOING and SPI_IFACE_LOG_INCOMING - used to enable logging of transactions on the AT layer.
 - #define SPI_IFACE_ADD_TRAILING_RN - if set to 1, the driver adds "\r\n" string to the end of commands. If 0, user application needs to pass commands with this string included.
 - #define SPI_IFACE_TIMEOUT_DEF_MS and SPI_IFACE_TIMEOUT_INIT_MS - used to specify maximum timeout for the reception of the first message after NCP initialization and subsequent responses.
- spi_iface.c
 - spi_iface_lock - variable used as a mutex signifying an ongoing command / response transaction. When a command is sent, this is set to 1 to make sure any incoming message is taken as a response to this command.
 - spi_iface_init - initializes the NCP, exchanges the first AT command / response to verify it state.
 - spi_iface_deinit - deinitializes the NCP.
 - spi_iface_command - sends an AT command and collects the response.
 - spi_iface_data - sends a data payload with a header frame.
 - spi_iface_receive_report - receives a report from the NCP. To be called by the application when the resources allow to receive.
 - spi_iface_ncp_ready_high - interrupt function raised when the SPI_RDY pin is pulled high. If there is no ongoing command / response transaction, this is considered as an incoming report and signalled to the application.

5. Porting to other boards

Porting to other boards should be done by modifying the low-level SPI driver (another SPI, different speed, etc.,) and pin allocations to reflect the pin mapping between the given host and ST67 NCP.

The necessary changes are mostly located in main.c/.h, spi_port.h and stm32g0xx_it.c files.

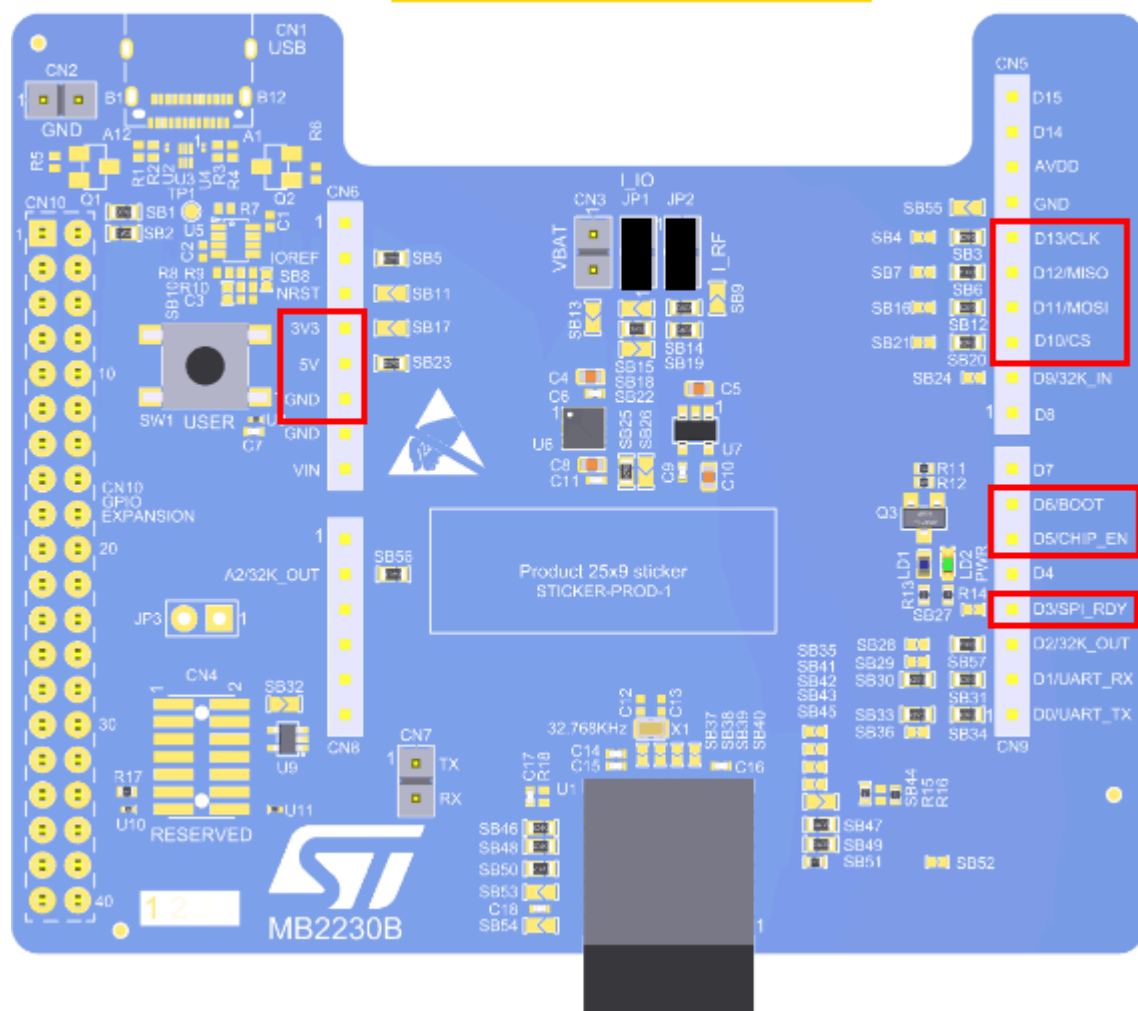
The module pinout can be found in [UM3449](#) in Section 8.3 "ARDUINO® Uno V3 connector".

The minimal list of pins which need to be connected between the host and NCP (expected name aliases are added):

- D13 - SPI_SCK

- D12 - SPI_MOSI
- D11 - SPI_MISO
- D10 - SPI_CS
- D6 - BOOT
- D5 - CHIP_EN
- D3 - SPI_RDY
- 3V3
- 5V
- GND

X-NUCLEO-67W61M1



Highlight of the used pins on the X-Nucleo board.

UART and low speed clock signal are not included here as that functionality is not used in the project.

The high-level interface files (spi_port.c and spi_iface.c/.h) can be directly copied.

6. Known limitations

On the C0 host, the SPI speed is limited to 6 MHz.

The driver has been tested with NCP binary v1.0.0. The main difference is in the list of possible report messages and usage of several commands.

The script to update the ST67 FW binary (originally located inside Projects\ST67W6X_Uutilities\Binaries) is not ported. It is expected that the ST67 will have an up-to-date FW installed by using the base package prior to using this example.